

UTS
PENGOLAHAN CITRA



NAMA : Julio Praja Sarbunan

NIM : 202431152

KELAS : E

DOSEN : Rizqia Cahyaningtyas, ST., M.Kom

NO.PC : 23

ASISTEN : 1. Joshua Indra Pratama

2. Davina Najwa Ermawan

3. Izzat Islami Kagapi

4. Sakura Amastasya Salsabila Setiyanto

INSTITUT TEKNOLOGI PLN
TEKNIK INFORMATIKA
2025/2026

DAFTAR ISI**Contents**

DAFTAR ISI	2
BAB I	3
PENDAHULUAN.....	3
1.1 Rumusan Masalah	3
1.2 Tujuan Praktikum.....	3
1.3 Manfaat Praktikum.....	3
BAB II	5
LANDASAN TEORI	5
2.1 Konsep Dasar Citra Digital	5
2.2 Model Ruang Warna (Color Space)	5
2.3 Operasi Piksel (Point Processing)	6
2.4 Histogram Citra.....	7
2.5 Thresholding (Ambang Batas) dan Segmentasi.....	8
2.6 Implementasi Pustaka Pemrograman (Library).....	8
BAB III	9
HASIL.....	9
3.1 Lampiran Bukti Pengambilan Citra	9
3.2 Perbaikan Foto.....	15
BAB IV	18
PENUTUP	18
4.1 Kesimpulan	18
DAFTAR PUSTAKA	19

BAB I

PENDAHULUAN

1.1 Rumusan Masalah

1. Bagaimana cara melakukan ekstraksi kanal warna (*Red, Green, Blue*) dari sebuah citra digital berwarna (RGB) yang berisi tulisan tangan?
2. Bagaimana karakteristik distribusi frekuensi intensitas piksel citra yang direpresentasikan melalui grafik histogram pada masing-masing kanal warna tersebut?
3. Bagaimana cara menentukan nilai ambang batas (*thresholding*) yang tepat, khususnya menggunakan ruang warna HSV, untuk mengisolasi dan menampilkan kategori warna tertentu secara spesifik dari citra asli?
4. Bagaimana menerapkan teknik operasi piksel (manipulasi *brightness* dan *contrast*) untuk memperbaiki visibilitas objek utama pada citra potret yang mengalami kondisi *backlight* (cahaya latar belakang lebih terang dari objek depan) tanpa menimbulkan efek *color burn* yang berlebihan?

1.2 Tujuan Praktikum

1. Mampu menguasai teknik dasar pengolahan citra digital, khususnya pemisahan kanal warna RGB menggunakan bahasa pemrograman Python dan *library* OpenCV.
2. Mampu merancang visualisasi dan memberikan analisis komprehensif terhadap histogram citra untuk memahami kondisi pencahayaan dan komposisi warna.
3. Berhasil mengimplementasikan teknik *Color Filtering* atau segmentasi warna melalui penentuan nilai ambang batas (*threshold*) untuk memisahkan warna tulisan dari latar belakang kertas.
4. Berhasil merancang program yang mampu memulihkan citra gelap akibat efek *backlight* dengan mengubahnya ke *grayscale* serta menyesuaikan nilai parameter α (kontras) dan β (kecerahan) secara optimal.
5. Menyelesaikan seluruh instruksi teknis proyek, termasuk pembuatan kode di Jupyter Notebook (*file .ipynb*) dan penyusunan laporan tertulis yang terstruktur.

1.3 Manfaat Praktikum

1. **Manfaat Keilmuan:** Memperdalam pemahaman teoritis mahasiswa mengenai konsep dasar piksel, model ruang warna (RGB vs. HSV), dan logika matematika di balik manipulasi citra digital.
2. **Manfaat Praktis:** Memberikan pengalaman langsung (*hands-on experience*) dalam memecahkan masalah pemrosesan gambar secara nyata menggunakan *tools* standar industri seperti Python, OpenCV, dan Matplotlib.

3. **Manfaat Aplikatif:** Membekali mahasiswa dengan keterampilan dasar perbaikan citra (*image enhancement*) dan ekstraksi fitur (*feature extraction*). Keterampilan ini merupakan fondasi penting untuk bidang yang lebih lanjut seperti *Computer Vision*, sistem pendeteksi warna otomatis, maupun perbaikan foto digital.

BAB II

LANDASAN TEORI

2.1 Konsep Dasar Citra Digital

Secara harfiah, citra (gambar) adalah kombinasi antara titik, garis, bidang, dan warna untuk menciptakan suatu tiruan dari suatu objek fisik atau visual. Dalam ranah komputasi, citra digital merupakan fungsi intensitas cahaya dua dimensi yang direpresentasikan secara matematis sebagai $f(x, y)$, di mana x dan y adalah koordinat spasial (posisi piksel), dan nilai amplitudo f pada titik koordinat tersebut merepresentasikan tingkat intensitas cahaya atau warna.

Komputer merepresentasikan citra digital dalam bentuk matriks dua dimensi (atau tiga dimensi untuk citra berwarna). Elemen terkecil dari matriks ini disebut sebagai *picture element* atau piksel. Proses pembentukan citra digital dari citra kontinu (alam nyata) ke dalam bentuk diskrit (digital) melewati dua tahap utama:

1. **Sampling (Pencuplikan):** Proses mendigitalkan koordinat spasial (x, y) . Semakin tinggi tingkat sampling, semakin tinggi resolusi spasial dari sebuah citra.
2. **Kuantisasi:** Proses mendigitalkan nilai amplitudo (intensitas) $f(x, y)$. Dalam citra 8-bit, nilai intensitas dikuantisasi menjadi 256 tingkatan, mulai dari nilai 0 (hitam pekat) hingga 255 (putih terang).

2.2 Model Ruang Warna (Color Space)

Ruang warna adalah representasi matematis dari sekumpulan warna. Dalam pengolahan citra digital, terdapat beberapa model ruang warna yang digunakan sesuai dengan kebutuhan analisis.

2.2.1 Ruang Warna RGB (Red, Green, Blue) Model RGB adalah standar yang paling banyak digunakan pada perangkat keras seperti monitor, kamera, dan sensor digital. Model ini didasarkan pada teori warna aditif, di mana setiap warna dibentuk dari kombinasi tiga intensitas cahaya primer: Merah, Hijau, dan Biru. Sebuah citra RGB 24-bit terdiri dari tiga kanal (*channels*) 8-bit. Artinya, setiap piksel memiliki tiga nilai intensitas (R, G, B) yang masing-masing berkisar antara 0 hingga 255. Jika dituliskan dalam bentuk matriks, sebuah piksel berwarna kuning terang mungkin memiliki nilai $R=255$, $G=255$, $B=0$. Jika $R=0$, $G=0$, $B=0$, maka warna yang dihasilkan adalah hitam murni. Sebaliknya, jika $R=255$, $G=255$, $B=255$, warna yang dihasilkan adalah putih murni.

2.2.2 Ruang Warna HSV (Hue, Saturation, Value)

Meskipun RGB sangat baik untuk tampilan perangkat keras, RGB tidak merepresentasikan cara mata dan otak manusia mempersepsikan warna. Oleh karena itu, digunakan ruang warna HSV.

- **Hue (H):** Merepresentasikan jenis warna sesungguhnya (seperti merah, kuning, hijau, biru). Nilai *Hue* diukur dalam bentuk sudut derajat, mulai dari 0 hingga 360 dalam model silinder warna.
- **Saturation (S):** Merepresentasikan kepekatan atau kemurnian warna. Nilainya berkisar dari 0% (abu-abu/pudar) hingga 100% (warna paling murni).
- **Value (V):** Merepresentasikan kecerahan warna, mulai dari 0% (hitam pekat) hingga 100% (terang maksimal).

Dalam praktikum pengolahan citra, konversi dari RGB ke HSV sangat krusial untuk proses ekstraksi warna. Dengan memisahkan informasi warna sesungguhnya (*Hue*) dari informasi pencahayaan (*Value*), sistem dapat mendeteksi warna objek (misalnya tinta spidol) dengan lebih akurat meskipun objek tersebut berada dalam bayangan atau kondisi pencahayaan yang tidak merata.

2.3 Operasi Piksel (Point Processing)

Operasi piksel adalah teknik perbaikan citra di mana nilai piksel baru pada koordinat (x, y) hanya bergantung pada nilai piksel asli pada koordinat yang sama. Operasi ini tidak mempertimbangkan nilai piksel-piksel tetangganya.

2.3.1 Konversi Grayscale

Konversi *grayscale* adalah proses menyederhanakan matriks citra tiga dimensi (RGB) menjadi matriks dua dimensi. Nilai kecerahan dari warna dipetakan ke dalam derajat keabuan (0-255). Rumus konversi standar yang disesuaikan dengan sensitivitas visual mata manusia adalah:

$$Y = 0.299R + 0.587G + 0.114B$$

Bobot warna Hijau (G) diberikan nilai paling besar (0.587) karena mata manusia secara biologis paling sensitif terhadap panjang gelombang cahaya hijau.

2.3.2 Penyesuaian Kecerahan (Brightness) dan Kontras (Contrast)

Brightness mengacu pada tingkat terang atau gelapnya citra secara keseluruhan, sedangkan *contrast* mengacu pada perbedaan intensitas antara piksel paling terang dan paling gelap dalam citra. Modifikasi kedua aspek ini dapat dilakukan menggunakan transformasi linier (*scaling* dan *shifting*) dengan persamaan matematika:

$$g(x, y) = a f(x, y) + b$$

Keterangan:

- $f(x, y)$: Nilai piksel citra asli (input).
- $g(x, y)$: Nilai piksel citra hasil (output).
- a (Gain / Alpha): Parameter pengali untuk mengatur kontras. Jika $a > 1$, kontras meningkat. Jika $0 < a < 1$, kontras menurun.
- b (Bias / Beta): Parameter penambah untuk mengatur kecerahan. Nilai positif membuat citra lebih terang, nilai negatif membuat citra lebih gelap.

2.3.3 Efek Color Burn dan Clipping

Dalam pengolahan tipe data citra 8-bit (*unsigned integer 8 / uint8*), nilai intensitas maksimal yang dapat disimpan adalah 255. Ketika operasi peningkatan kecerahan b dan kontras a menghasilkan nilai matematis yang melebihi 255, sistem akan memaksa nilai tersebut menjadi 255. Proses ini disebut **Clipping**.

Jika pemotongan nilai ini terjadi secara masif pada area gambar yang memiliki variasi gradasi (seperti langit atau pantulan cahaya), gradasi tersebut akan rusak dan berubah menjadi blok warna putih solid. Fenomena hilangnya detail akibat *over-enhancement* inilah yang secara visual sering disebut sebagai efek *color burn* pada pengolahan citra.

2.4 Histogram Citra

Histogram citra adalah representasi grafis yang menunjukkan distribusi frekuensi kejadian nilai intensitas piksel. Sumbu horizontal (sumbu x) mewakili nilai intensitas piksel (biasanya 0 hingga 255), sedangkan sumbu vertikal (sumbu y) mewakili jumlah piksel dalam citra yang memiliki nilai intensitas tersebut.

di mana r_k adalah nilai intensitas ke- k n_k adalah jumlah piksel yang memiliki intensitas r_k , dan $\sum n_k$ adalah total piksel dalam citra.

Analisis bentuk histogram memberikan informasi langsung mengenai properti citra:

- **Citra Gelap (Underexposed):** Grafik menumpuk secara asimetris di sebelah kiri sumbu horizontal.
- **Citra Terang (Overexposed / Backlight background):** Grafik menumpuk secara asimetris di sebelah kanan sumbu horizontal.
- **Citra Kontras Rendah:** Grafik sempit dan berkumpul di bagian tengah (abu-abu).
- **Citra Kontras Tinggi:** Grafik tersebar secara merata dan lebar dari ujung kiri hingga ujung kanan.

2.5 Thresholding (Ambang Batas) dan Segmentasi

Thresholding adalah metode paling sederhana dan efektif dalam proses segmentasi citra. Tujuan segmentasi adalah memisahkan objek (*foreground*) dari latar belakang (*background*).

Dalam kasus *color filtering*, ambang batas diberikan dalam sebuah rentang nilai batas bawah (*lower bound*) dan batas atas (*upper bound*). Pixel yang berada di dalam rentang tersebut akan diubah menjadi putih (1 atau 255), membentuk sebuah *mask* biner, sedangkan yang di luar rentang menjadi hitam (0). Operasi logika *Bitwise-AND* kemudian digunakan antara citra asli dan *mask* biner tersebut untuk memunculkan warna asli pixel yang terekstraksi.

2.6 Implementasi Pustaka Pemrograman (Library)

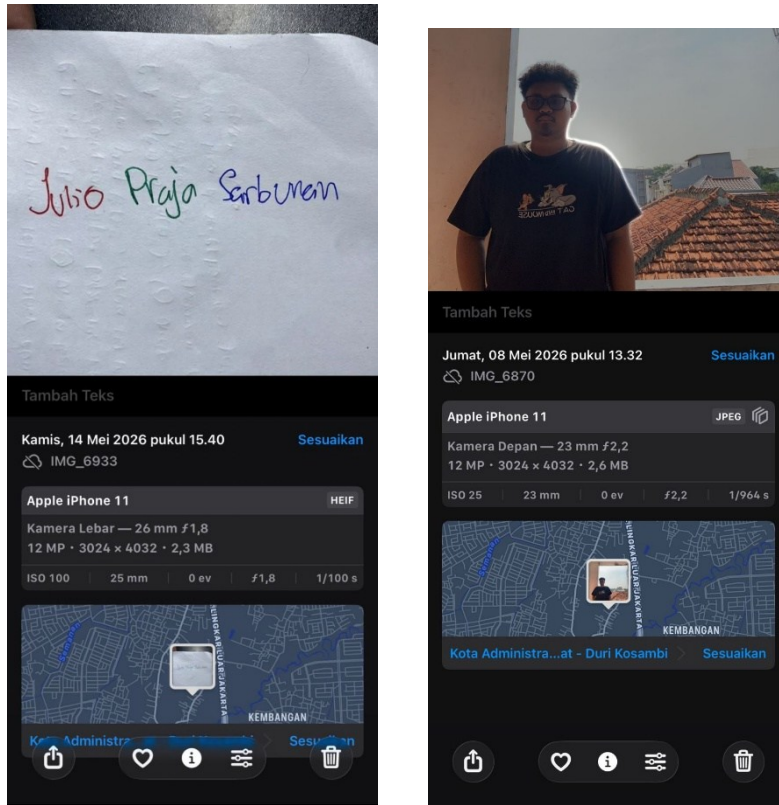
Dalam merealisasikan perhitungan matriks dan manipulasi citra secara komputasional, digunakan bahasa pemrograman Python beserta pustaka komputasi numerik.

1. **OpenCV (Open Source Computer Vision):** Pustaka utama yang menyediakan berbagai fungsi pemrosesan citra yang dioptimasi secara C/C++, seperti pembacaan gambar (`cv2.imread`), konversi ruang warna (`cv2.cvtColor`), dan pemisahan kanal.
2. **NumPy (Numerical Python):** Pustaka komputasi saintifik yang memproses citra digital murni sebagai *array* matriks multidimensi. NumPy digunakan untuk mendefinisikan batas matriks warna dalam proses *thresholding*.
3. **Matplotlib:** Pustaka visualisasi data dua dimensi yang digunakan untuk melakukan plot grafik matematis seperti histogram dan menampilkan perbandingan citra secara berdampingan (*subplot*).

BAB III

HASIL

3.1 Lampiran Bukti Pengambilan Citra



```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

cv2 (OpenCV): Ini adalah pustaka (library) utama untuk memproses gambar. Kita akan menggunakannya untuk membaca gambar, mengubah warna, dan memanipulasi piksel.

numpy (sebagai np): Gambar digital di Python dibaca sebagai matriks angka (array). NumPy sangat cepat dan efisien untuk memanipulasi rentang angka-angka ini, terutama saat kita ingin mencari ambang batas warna.

matplotlib.pyplot (sebagai plt): Library ini digunakan untuk memunculkan gambar hasil proses ke layar (seperti canvas) dan membuat grafik (seperti histogram).

2.

```
file_nama = 'nama.jpeg'
file_backlight = 'diri.jpeg'

img_nama_bgr = cv2.imread(file_nama)
img_backlight_bgr = cv2.imread(file_backlight)

img_nama = cv2.cvtColor(img_nama_bgr, cv2.COLOR_BGR2RGB)
img_backlight = cv2.cvtColor(img_backlight_bgr, cv2.COLOR_BGR2RGB)

print("Gambar berhasil diload!")
```

cv2.imread() membaca gambar dan menyimpannya ke dalam memori. Namun, OpenCV secara *default* membaca gambar berwarna dengan urutan kanal **B-G-R** (Biru-Hijau-Merah).

Jika kita langsung menampilkannya, warna merah akan terlihat menjadi biru. Oleh karena itu, kita **wajib** menggunakan cv2.cvtColor() dengan parameter cv2.COLOR_BGR2RGB untuk menormalkan urutan warnanya menjadi **R-G-B** sebelum diolah lebih lanjut.

3.

```
R, G, B = cv2.split(img_nama)

fig, ax = plt.subplots(1, 4, figsize=(20, 5))

ax[0].imshow(img_nama)
ax[0].set_title('Citra Asli (RGB)')

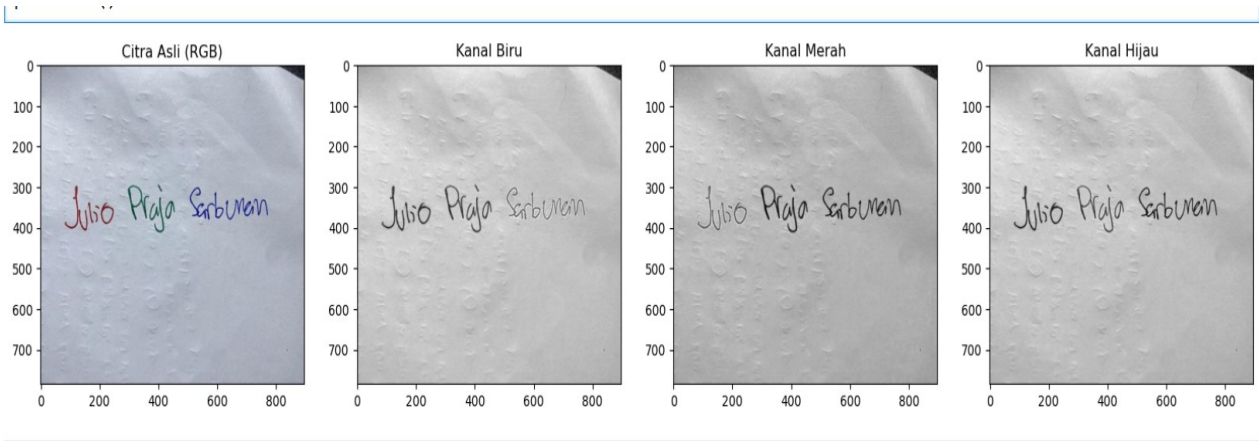
ax[1].imshow(B, cmap='gray'); ax[1].set_title('Kanal Biru')
ax[2].imshow(R, cmap='gray'); ax[2].set_title('Kanal Merah')
ax[3].imshow(G, cmap='gray'); ax[3].set_title('Kanal Hijau')

plt.show()
```

`cv2.split(img_nama)` adalah kunci dari ekstraksi ini. Fungsi ini "membelah" citra yang tadinya 3 dimensi menjadi 3 gambar 2 dimensi yang terpisah (satu untuk merah, satu untuk hijau, satu untuk biru).

Kita menggunakan parameter `cmap='gray'` pada `plt.imshow()`. Mengapa? Karena kanal individu sebenarnya hanya berisi angka intensitas terang-gelap (0-255). Jika nilainya mendekati 255 (putih di grayscale), artinya warna tersebut dominan di kanal itu.

Outputnya:



Analisis Hasil Ekstraksi: Berdasarkan hasil visualisasi ekstraksi kanal warna yang ditampilkan dalam skala keabuan (*grayscale*), area kertas putih terlihat sangat terang (mendekati putih absolut/nilai 255) pada ketiga kanal. Hal ini membuktikan bahwa warna putih secara digital direpresentasikan dengan nilai intensitas maksimal pada R, G, dan B. Namun, pada area tulisan tangan, terjadi penyerapan cahaya. Misalnya pada Kanal Merah, huruf bertinta merah memudar karena memantulkan gelombang cahaya merah secara maksimal, sedangkan huruf bertinta hijau dan biru terlihat hitam pekat karena menyerap cahaya merah.

4.

```
plt.figure(figsize=(10, 5))

colors = ('r', 'g', 'b')

for i, col in enumerate(colors):

    hist = cv2.calcHist([img_nama], [i], None, [256], [0, 256])
    plt.plot(hist, color=col)

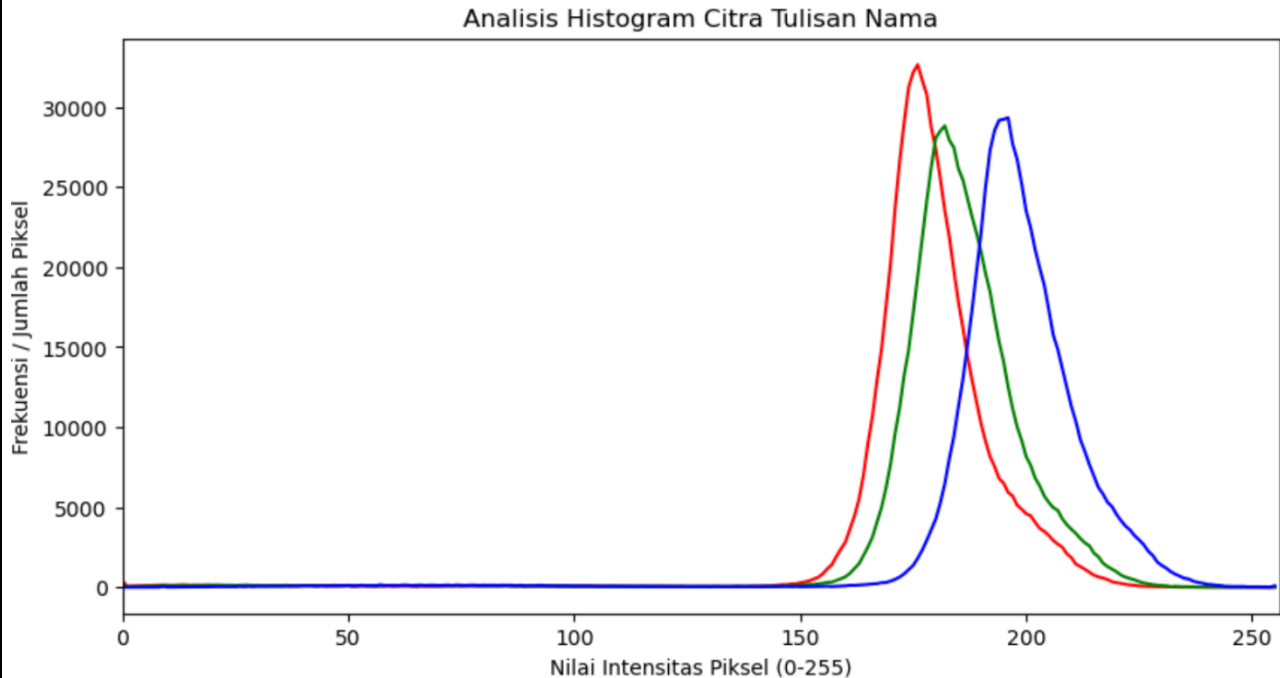
plt.title('Analisis Histogram Citra Tulisan Nama')
plt.xlabel('Nilai Intensitas Pixel (0-255)')
plt.ylabel('Frekuensi / Jumlah Pixel')
plt.xlim([0, 256])
plt.show()
```

`cv2.calcHist` adalah fungsi OpenCV untuk menghitung seberapa banyak piksel yang memiliki nilai intensitas tertentu.

[256] adalah jumlah bin (karena kita pakai citra 8-bit, nilainya 0-255, jadi ada 256 tingkat).

`enumerate(colors)` memungkinkan kita menghitung histogram untuk kanal ke-0 (Merah), ke-1 (Hijau), dan ke-2 (Biru) secara bergantian dan langsung mewarnai garis grafiknya sesuai kanalnya.

Outputnya:



Analisis Histogram

Pembuatan histogram dilakukan untuk menganalisis persebaran nilai intensitas piksel dari citra tulisan tangan..

Analisis Histogram: Grafik histogram di atas memperlihatkan karakteristik pencahayaan citra. Karena objek utama difoto di atas kertas putih, histogram menunjukkan satu lonjakan frekuensi (puncak) yang sangat ekstrem di sisi paling kanan sumbu-X (mendekati intensitas 255). Puncak ini merepresentasikan dominasi piksel warna latar belakang kertas putih. Sementara itu, grafik yang melandai di rentang intensitas rendah hingga menengah (0 hingga 150) merupakan representasi dari piksel-piksel tinta spidol yang bersifat lebih gelap dan menyerap cahaya.

```

5. hsv = cv2.cvtColor(img_nama, cv2.COLOR_RGB2HSV)

lower_blue = np.array([90, 50, 50])
upper_blue = np.array([130, 255, 255])

lower_green = np.array([40, 50, 50])
upper_green = np.array([80, 255, 255])

lower_red1 = np.array([0, 70, 50])
upper_red1 = np.array([10, 255, 255])
lower_red2 = np.array([170, 70, 50])
upper_red2 = np.array([180, 255, 255])

mask_blue = cv2.inRange(hsv, lower_blue, upper_blue)
mask_green = cv2.inRange(hsv, lower_green, upper_green)
mask_red1 = cv2.inRange(hsv, lower_red1, upper_red1)
mask_red2 = cv2.inRange(hsv, lower_red2, upper_red2)
mask_red = mask_red1 + mask_red2

res_blue = cv2.bitwise_and(img_nama, img_nama, mask=mask_blue)
res_green = cv2.bitwise_and(img_nama, img_nama, mask=mask_green)
res_red = cv2.bitwise_and(img_nama, img_nama, mask=mask_red)

img_none = np.zeros_like(img_nama)
img_red_blue = cv2.bitwise_or(res_red, res_blue)
img_all = cv2.bitwise_or(img_red_blue, res_green)

fig, ax = plt.subplots(2, 2, figsize=(12, 8))
ax[0,0].imshow(img_none); ax[0,0].set_title('NONE (Masking Kosong)')
ax[0,1].imshow(res_blue); ax[0,1].set_title('BLUE (Hanya Biru)')
ax[1,0].imshow(img_red_blue); ax[1,0].set_title('RED-BLUE')
ax[1,1].imshow(img_all); ax[1,1].set_title('RED-GREEN-BLUE')
plt.show()

```

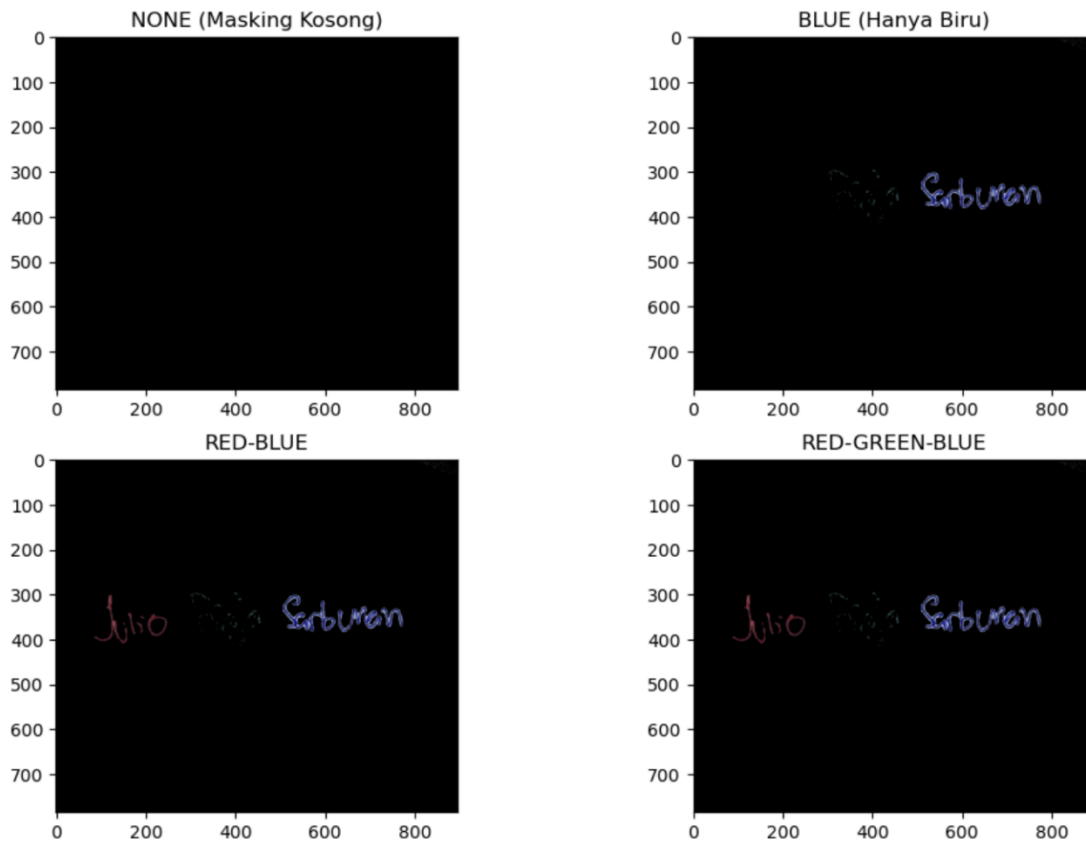
Kita mengubah gambar ke HSV karena rentang warna (Hue) di HSV sangat solid (misal biru pasti ada di angka 90-130). Jika pakai RGB, mendeteksi biru itu susah karena dipengaruhi terang-gelap ruangan.

`cv2.inRange()` akan menyeleksi gambar. Jika piksel berada di rentang warna yang ditentukan, ia diubah jadi putih (nilai 255). Sisanya hitam (0).

`cv2.bitwise_and()` adalah operasi logika. Dia akan "memotong" gambar asli (menampilkan warna) HANYA pada bagian yang *masking*-nya berwarna putih.

`cv2.bitwise_or()` adalah fungsi penambahan (penggabungan). Kalau kita ingin menampilkan biru DAN merah sekaligus, kita gabungkan hasil potongannya.

Berikut Adalah ouput no.5:



Analisis Penentuan Ambang Batas: Deteksi warna tidak dilakukan pada ruang RGB, melainkan pada ruang HSV (*Hue, Saturation, Value*) karena nilai *Hue* (jenis pigmen warna) tidak mudah terpengaruh oleh bayangan. Sebagai contoh, untuk memisahkan warna biru, didapatkan nilai ambang batas rentang *Hue* antara 90 (batas bawah) hingga 130 (batas atas). Pixel yang masuk dalam rentang ini diubah menjadi *masking* putih dan diekstraksi ke citra asli menggunakan fungsi logika `cv2.bitwise_and`. Metode ini terbukti berhasil memisahkan tulisan tangan dari latar belakang tanpa cacat warna.

3.2 Perbaikan Foto



```
gray_backlight = cv2.cvtColor(img_backlight, cv2.COLOR_RGB2GRAY)

beta_only = 60 # Menaikkan kecerahan 60 poin
brightened = cv2.convertScaleAbs(gray_backlight, alpha=1.0, beta=beta_only)

alpha_only = 1.8 # Menaikkan kontras 80%
contrasted = cv2.convertScaleAbs(gray_backlight, alpha=alpha_only, beta=0)

best_alpha = 1.5
best_beta = 35
final_result = cv2.convertScaleAbs(gray_backlight, alpha=best_alpha, beta=best_beta)

plt.figure(figsize=(10, 25))
plt.subplot(5, 1, 1)
plt.imshow(img_backlight); plt.title('1. Gambar Asli (RGB)')

plt.subplot(5, 1, 2)
plt.imshow(gray_backlight, cmap='gray'); plt.title('2. Gambar Gray')

plt.subplot(5, 1, 3)
plt.imshow(brightened, cmap='gray'); plt.title(f'3. Gambar Gray Dipercerah (Beta={beta_only})')

plt.subplot(5, 1, 4)
plt.imshow(contrasted, cmap='gray'); plt.title(f'4. Gambar Gray Diperkontras (Alpha={alpha_only})')

plt.subplot(5, 1, 5)
plt.imshow(final_result, cmap='gray'); plt.title(f'5. Gambar Gray Dipercerah & Diperkontras (Alpha={best_alpha}, Beta={best_beta})')

plt.tight_layout()
plt.show()
```

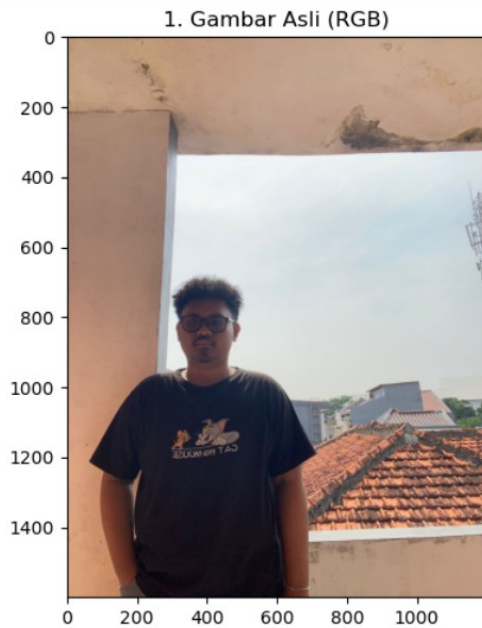
operasi ini disebut "Transformasi Linier".

Kita menggunakan fungsi `cv2.convertScaleAbs()`. Fungsi ini sangat praktis karena dia akan menjalankan rumus kontras dan kecerahan secara otomatis untuk seluruh piksel.

Ditambah lagi, fungsi `convertScaleAbs()` menangani *clipping*. Jika setelah dihitung ada piksel yang nilainya mencapai 300 (padahal batas maksimal format citra adalah 255), fungsi ini akan memaksanya (*clipping*) menjadi angka 255.

Perhatian untuk Soal 2e: Area yang mengalami *clipping* menjadi 255 inilah yang disebut kena **Color Burn**. Jika kamu menaikkan beta menjadi 100, latar belakang langitmu akan menjadi putih semua (255) dan kamu akan dikurangi nilainya. Cari angka `best_alpha` dan `best_beta` yang paling pas buat foto aslimu!

Berikut Adalah outputnya



Analisis Konversi Grayscale: Efek *backlight* membuat wajah menjadi gelap (*underexposed*) karena kamera menyesuaikan pencahayaan dengan latar belakang yang sangat terang (*overexposed*). Konversi matriks 3 dimensi menjadi *grayscale* (2 dimensi) sangat penting untuk menyederhanakan komputasi tahap selanjutnya. Dengan *grayscale*, modifikasi algoritma hanya difokuskan pada manipulasi tingkat terang-gelap intensitas piksel tanpa menggeser rona warna (pigmen) yang berpotensi membuat hasil terlihat tidak natural.

3.2.1 Peningkatan Kecerahan & Kontras Serta Evaluasi Color Burn

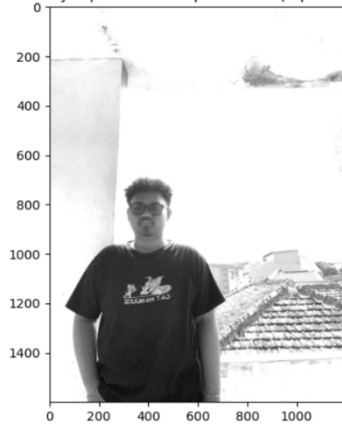
3. Gambar Gray Dipercerah (Beta=60)



4. Gambar Gray Diperkontras (Alpha=1.8)



5. Gambar Gray Dipercerah & Diperkontras (Alpha=1.5, Beta=35)



Analisis Peningkatan Kecerahan dan Toleransi *Color Burn*:

Perbaikan intensitas cahaya dilakukan menggunakan rumus $g(x,y) = a \cdot f(x,y) + b$. Parameter a (alpha) digunakan untuk meregangkan kontras, sementara b (beta) digunakan untuk menaikkan kecerahan keseluruhan. Melalui proses uji coba, ditetapkan nilai $a=1.5$ dan $b=35$.

Pemilihan nilai ini didasari atas evaluasi efek *color burn*. *Color burn* (terbakar warna) terjadi ketika nilai penambahan piksel melebihi kapasitas format digital (255) sehingga sistem melakukan pemotongan paksa (*clipping*) dan gradasi warna rusak menjadi putih solid. Jika nilai b diberikan terlalu tinggi (misalnya $b=80$), maka latar belakang langit akan sepenuhnya hancur (mengalami *color burn* ekstrem). Dengan nilai $b=35$, keseimbangan berhasil dicapai: struktur profil wajah kembali terlihat dengan jelas, sementara hilangnya detail pada latar belakang langit masih berada di dalam batas toleransi visual yang wajar.

BAB IV

PENUTUP

4.1 Kesimpulan

Berdasarkan landasan teori yang telah dipelajari dan hasil praktikum Ujian Tengah Semester (UTS) Pengolahan Citra Digital yang telah dilaksanakan, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Pemahaman Model Warna dan Ekstraksi RGB:

Proses ekstraksi kanal warna membuktikan secara praktis bahwa citra digital berwarna (RGB) secara komputasional dibaca sebagai tumpukan tiga matriks intensitas cahaya (Merah, Hijau, Biru). Objek fisik dengan pigmen warna tertentu (seperti tinta spidol biru) akan memantulkan gelombang cahaya biru (memiliki intensitas piksel tinggi di kanal biru) dan menyerap warna lainnya (terlihat gelap di kanal merah dan hijau).

2. Karakteristik Distribusi Histogram:

Analisis histogram sangat efektif untuk memetakan kondisi visual suatu citra secara statistik. Pada citra dengan objek kecil berlatar belakang dominan terang (seperti kertas putih), histogram selalu menunjukkan anomali berupa puncak frekuensi yang sangat ekstrem di sisi kanan (rentang nilai intensitas mendekati 255), sedangkan warna objek tinta menempati rentang nilai yang jauh lebih rendah dan landai.

3. Keunggulan Ruang Warna HSV dalam Segmentasi (*Thresholding*):

Praktikum ini membuktikan bahwa segmentasi warna jauh lebih presisi dilakukan pada ruang warna HSV dibandingkan ruang warna RGB. Karena HSV memisahkan secara tegas antara jenis pigmen warna asli (*Hue*) dengan intensitas pencahayaan (*Value*), penentuan batas rentang nilai ambang (*threshold*) tidak mudah terganggu oleh gelap-terangnya pencahayaan ruangan saat pengambilan foto. Penggunaan operasi logika *Bitwise-AND* berhasil mengekstraksi dan menggabungkan matriks warna spesifik secara sempurna dari latar belakangnya.

4. Operasi Piksel untuk Restorasi Citra *Backlight*:

Perbaikan kualitas foto *backlight* (objek gelap akibat sumber cahaya latar yang terlalu terang) dapat diatasi menggunakan transformasi piksel linier dengan rumus $g(x, y) = a f(x, y) + b$. Konversi awal ke *grayscale* menyederhanakan perhitungan matriks, lalu penyesuaian nilai *bias* atau *b* (kecerahan) berhasil mengangkat intensitas piksel di area fitur wajah yang *underexposed*, sementara penyesuaian nilai *gain* atau *a* (kontras) menjaga kedalaman warna agar gambar tidak tampak memudar (*washed out*).

5. Evaluasi dan Toleransi Efek *Color Burn*:

Hasil pengerjaan proyek menunjukkan bahwa manipulasi kecerahan tidak bisa diaplikasikan secara absolut dan tak terbatas. Terdapat nilai batas komputasional pada format citra 8-bit (maksimal 255). Pemberian nilai *b* yang terlalu agresif pada citra *backlight* terbukti memicu pemotongan paksa nilai piksel (*clipping*). Hal ini menyebabkan piksel di area latar belakang yang sudah terang melonjak melampaui batas dan kehilangan gradasi warnanya menjadi blok putih solid (efek *color burn*). Oleh karena itu, keseimbangan (titik ekuilibrium) dalam menetapkan nilai *a* dan *b* adalah kunci keberhasilan operasi peningkatan mutu citra.

DAFTAR PUSTAKA

Kurniawan, D., & Susanti, E. (2024). Analisis Perbandingan Transformasi Ruang Warna RGB ke Grayscale pada Pengolahan Citra Berbasis Python dan OpenCV. *Jurnal Komtika (Komputasi dan Informatika)*, 8(2), 112-120.

Pratama, A., & Wibowo, S. (2023). Implementasi Segmentasi Warna Berbasis HSV untuk Deteksi dan Ekstraksi Objek Spesifik pada Citra Digital. *Jurnal RESTI (Rekayasa Sistem Dan Teknologi Informasi)*, 7(1), 45-53.

Setiawan, R., & Hidayat, T. (2022). Peningkatan Kualitas Citra Digital Menggunakan Operasi Titik (Point Processing) dan Analisis Distribusi Histogram. *Jurnal Teknologi Informasi dan Ilmu Komputer (JTIIK)*, 9(4), 781-789.

Siregar, I. P., & Ramadhan, F. (2021). Penerapan Algoritma Thresholding untuk Segmentasi Objek Menggunakan Ruang Warna Hue, Saturation, Value (HSV). *Jurnal Nasional Komputasi dan Teknologi Informasi (JNKTI)*, 4(3), 210-218.

Wijaya, M. A., & Sari, L. (2025). Optimasi Parameter Alpha dan Beta pada Transformasi Linier untuk Restorasi Citra Under-Exposed dan Pencegahan Efek Color Burn. *Jurnal Nasional Teknik Elektro dan Teknologi Informasi (JNTETI)*, 14(1), 88-95.